

Лабораторная работа №12

Алгоритм Дейкстры. Алгоритм Флойда-Уоршелла

Цель работы:

1. Реализовать алгоритм Дейкстры для нахождения кратчайших путей от одной вершины до всех остальных во взвешенном графе с неотрицательными весами
2. Реализовать алгоритм Флойда-Уоршелла для нахождения кратчайших путей между всеми парами вершин
3. Изучить особенности работы алгоритмов с графами, содержащими отрицательные веса
4. Научиться восстанавливать кратчайшие пути после выполнения алгоритмов
5. Сравнить временную сложность и области применения алгоритмов

Теоретическое обоснование

1. Постановка задачи

Задача поиска кратчайшего пути - одна из классических задач теории графов. Требуется найти путь между двумя вершинами графа, сумма весов рёбер которого минимальна.

Алгоритм	Назначение	Сложность	Отрицательные веса
Дейкстры	Из одной вершины до всех остальных	$O((V + E) \log V)$	Не допускаются
Флойда-Уоршелла	Между всеми парами вершин	$O(V^3)$	Допускаются (нет отрицательных циклов)

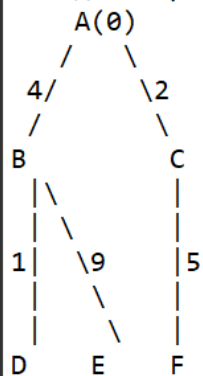
2. Алгоритм Дейкстры (Dijkstra's Algorithm)

Принцип работы: Жадный алгоритм, который на каждом шаге выбирает вершину с наименьшим известным расстоянием и релаксирует её соседей.

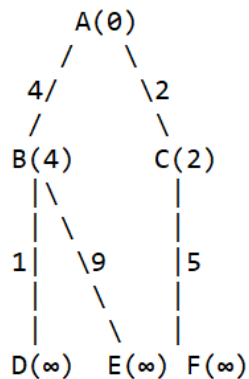
Условие применимости: Все веса рёбер должны быть неотрицательными.

Визуализация работы алгоритма Дейкстры:

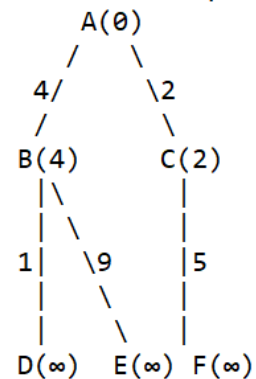
Исходный граф:



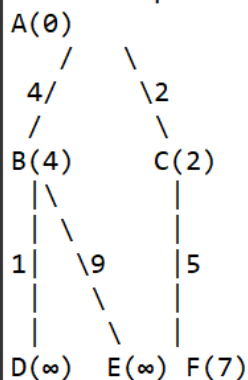
Шаг 1: Начало из A



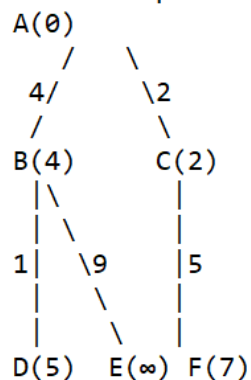
Шаг 2: Обработка A



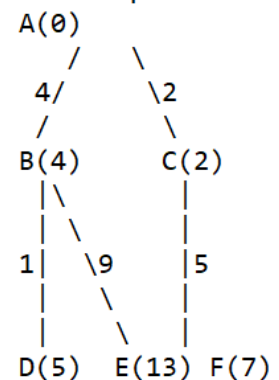
Шаг 3: Обработка C



Шаг 4: Обработка B



Шаг 5: Обработка D



Конечный результат: Кратчайшие расстояния от A: A=0, B=4, C=2, D=5, E=13, F=7

Алгоритм:

```

vector<int> dijkstra(int start) {
    vector<int> dist(V, INF);
    vector<bool> visited(V, false);

```

```

    dist[start] = 0;

```

```

    for (int i = 0; i < V - 1; i++) {

```

// Находим вершину с минимальным расстоянием среди необработанных

```

        int u = -1;

```

```

        for (int j = 0; j < V; j++) {

```

```

            if (!visited[j] && (u == -1 || dist[j] < dist[u])) {

```

```

                u = j;

```

```

            }

```

```

        }

```

```

        visited[u] = true;

```

```

// Релаксация рёбер
for (int v = 0; v < V; v++) {
    if (adj[u][v] != 0 && !visited[v]) {
        if (dist[u] + adj[u][v] < dist[v]) {
            dist[v] = dist[u] + adj[u][v];
        }
    }
}

return dist;
}

```

Оптимизированная версия с priority_queue:

```

vector<int> dijkstra_pq(int start) {
    vector<int> dist(V, INF);
    using pii = pair<int, int>; // {расстояние, вершина}
    priority_queue<pii, vector<pii>, greater<pii>> pq;

    dist[start] = 0;
    pq.push({0, start});

    while (!pq.empty()) {
        int d = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        if (d > dist[u]) continue; // устаревшая запись

        for (int v = 0; v < V; v++) {
            if (adj[u][v] != 0) {
                if (dist[u] + adj[u][v] < dist[v]) {
                    dist[v] = dist[u] + adj[u][v];
                    pq.push({dist[v], v});
                }
            }
        }
    }

    return dist;
}

```

Восстановление пути:

```
vector<int> dijkstraWithPath(int start, int end) {
    vector<int> dist(V, INF);
    vector<int> prev(V, -1);
    vector<bool> visited(V, false);

    dist[start] = 0;

    for (int i = 0; i < V - 1; i++) {
        int u = -1;
        for (int j = 0; j < V; j++) {
            if (!visited[j] && (u == -1 || dist[j] < dist[u])) {
                u = j;
            }
        }

        visited[u] = true;

        for (int v = 0; v < V; v++) {
            if (adj[u][v] != 0 && !visited[v]) {
                if (dist[u] + adj[u][v] < dist[v]) {
                    dist[v] = dist[u] + adj[u][v];
                    prev[v] = u;
                }
            }
        }
    }

    // Восстановление пути
    vector<int> path;
    for (int v = end; v != -1; v = prev[v]) {
        path.push_back(v);
    }
    reverse(path.begin(), path.end());
    return path;
}
```

3. Алгоритм Флойда-Уоршелла (Floyd-Warshall Algorithm)

Принцип работы: Динамическое программирование - на каждой итерации k разрешаем использовать вершины 0..k в качестве промежуточных.

Визуализация:

Матрица расстояний после каждой итерации k:

k = -1 (начальная):	k = 0:	k = 1:
[0, ∞, ∞, ∞]	[0, 1, 3, ∞]	[0, 1, 3, 4]
[1, 0, ∞, ∞]	[1, 0, 2, ∞]	[1, 0, 2, 3]
[∞, 2, 0, 5]	[3, 2, 0, 5]	[3, 2, 0, 4]
[∞, ∞, 5, 0]	[∞, ∞, 5, 0]	[4, 3, 4, 0]

k = 2:	k = 3 (конечная):
[0, 1, 3, 4]	[0, 1, 3, 4]
[1, 0, 2, 3]	[1, 0, 2, 3]
[3, 2, 0, 4]	[3, 2, 0, 4]
[4, 3, 4, 0]	[4, 3, 4, 0]

Алгоритм:

```
vector<vector<int>> floydWarshall() {  
    // Инициализация матрицы расстояний  
    vector<vector<int>> dist(V, vector<int>(V, INF));  
  
    for (int i = 0; i < V; i++) {  
        dist[i][i] = 0;  
        for (int j = 0; j < V; j++) {  
            if (adj[i][j] != 0) {  
                dist[i][j] = adj[i][j];  
            }  
        }  
    }  
  
    // Основной цикл алгоритма  
    for (int k = 0; k < V; k++) {  
        for (int i = 0; i < V; i++) {  
            for (int j = 0; j < V; j++) {  
                if (dist[i][k] != INF && dist[k][j] != INF) {  
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {  
                        dist[i][j] = dist[i][k] + dist[k][j];  
                    }  
                }  
            }  
        }  
    }  
  
    return dist;  
}
```

```
}
```

Обнаружение отрицательных циклов:

```
bool hasNegativeCycle() {  
    vector<vector<int>> dist = floydWarshall();  
  
    for (int i = 0; i < V; i++) {  
        if (dist[i][i] < 0) {  
            return true; // найден отрицательный цикл  
        }  
    }  
    return false;  
}
```

Восстановление путей:

```
vector<vector<int>> floydWarshallWithPath(vector<vector<int>>& next) {  
    vector<vector<int>> dist(V, vector<int>(V, INF));  
  
    // Инициализация next матрицы для восстановления путей  
    next.assign(V, vector<int>(V, -1));  
  
    for (int i = 0; i < V; i++) {  
        dist[i][i] = 0;  
        for (int j = 0; j < V; j++) {  
            if (adj[i][j] != 0) {  
                dist[i][j] = adj[i][j];  
                next[i][j] = j;  
            }  
        }  
    }  
  
    for (int k = 0; k < V; k++) {  
        for (int i = 0; i < V; i++) {  
            for (int j = 0; j < V; j++) {  
                if (dist[i][k] != INF && dist[k][j] != INF) {  
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {  
                        dist[i][j] = dist[i][k] + dist[k][j];  
                        next[i][j] = next[i][k];  
                    }  
                }  
            }  
        }  
    }  
}
```

```

    }

    return dist;
}

vector<int> getPath(int u, int v, const vector<vector<int>>& next) {
    if (next[u][v] == -1) return {};

    vector<int> path = {u};
    while (u != v) {
        u = next[u][v];
        path.push_back(u);
    }
    return path;
}

```

4. Сравнение алгоритмов

Характеристика	Алгоритм Дейкстры	Алгоритм Флойда-Уоршелла
Количество вершин	От одной до всех	Все пары
Сложность	$O((V+E) \log V)$ или $O(V^2)$	$O(V^3)$
Отрицательные веса	Нет	Да (без циклов)
Отрицательные циклы	Нет	Обнаруживает
Память	$O(V)$	$O(V^2)$
Релаксация	Жадная	Динамическое программирование
Применимость	Большие разреженные графы	Малые графы ($V \leq 500$)

Примеры выполнения практической части

Пример 1. Алгоритм Дейкстры (базовый уровень)

Задача: Реализовать алгоритм Дейкстры для нахождения кратчайших расстояний от заданной вершины.

```
#include <iostream>
#include <vector>
#include <climits>

using namespace std;

class Graph {
private:
    int V;
    vector<vector<int>> adj;

public:
    Graph(int vertices) : V(vertices) {
        adj.resize(V, vector<int>(V, 0));
    }

    void addEdge(int u, int v, int weight) {
        adj[u][v] = weight;
        adj[v][u] = weight; // для неориентированного графа
    }

    vector<int> dijkstra(int start) {
        vector<int> dist(V, INT_MAX);
        vector<bool> visited(V, false);

        dist[start] = 0;

        for (int i = 0; i < V - 1; i++) {
            // Находим вершину с минимальным расстоянием
            int u = -1;
            for (int j = 0; j < V; j++) {
                if (!visited[j] && (u == -1 || dist[j] < dist[u])) {
                    u = j;
                }
            }
        }
    }
};
```

```
        if (dist[u] == INT_MAX) break;    // остальные вершины
недостижимы
```

```
        visited[u] = true;
```

```
        // Релаксация
```

```
        for (int v = 0; v < V; v++) {
            if (adj[u][v] != 0 && !visited[v]) {
                if (dist[u] + adj[u][v] < dist[v]) {
                    dist[v] = dist[u] + adj[u][v];
                }
            }
        }
    }
}
```

```
return dist;
```

```
void printDistances(int start) {
```

```
    vector<int> dist = dijkstra(start);
```

```
    cout << "Кратчайшие расстояния от вершины " << start << ":" <<
```

```
endl
```

```
    for (int i = 0; i < V; i++) {
        if (dist[i] == INT_MAX) {
            cout << " До " << i << ": недостижима" << endl;
        } else {
            cout << " До " << i << ": " << dist[i] << endl;
        }
    }
}
};
```

```
int main() {
```

```
    Graph g(6);
```

```
    g.addEdge(0, 1, 4);
    g.addEdge(0, 2, 2);
    g.addEdge(1, 2, 1);
    g.addEdge(1, 3, 5);
    g.addEdge(2, 3, 8);
    g.addEdge(2, 4, 10);
```

```

g.addEdge(3, 4, 2);
g.addEdge(3, 5, 6);
g.addEdge(4, 5, 3);

g.printDistances(0);

return 0;
}

```

Ожидаемый вывод:

Кратчайшие расстояния от вершины 0:

```

До 0: 0
До 1: 3
До 2: 2
До 3: 8
До 4: 10
До 5: 13

```

Пример 2. Алгоритм Флойда-Уоршелла (средний уровень)

Задача: Реализовать алгоритм Флойда-Уоршелла для нахождения кратчайших путей между всеми парами вершин.

```

#include <iostream>
#include <vector>
#include <climits>

using namespace std;

class Graph {
private:
    int V;
    vector<vector<int>>> adj;

public:
    Graph(int vertices) : V(vertices) {
        adj.resize(V, vector<int>(V, 0));
    }

    void addEdge(int u, int v, int weight) {
        adj[u][v] = weight;
    }
}

```

```

vector<vector<int>> floydWarshall() {
    // Инициализация матрицы расстояний
    vector<vector<int>> dist(V, vector<int>(V, INT_MAX));

    for (int i = 0; i < V; i++) {
        dist[i][i] = 0;
        for (int j = 0; j < V; j++) {
            if (adj[i][j] != 0) {
                dist[i][j] = adj[i][j];
            }
        }
    }

    // Основной алгоритм
    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != INT_MAX && dist[k][j] != INT_MAX) {
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }
    }

    return dist;
}

void printAllPairsShortestPaths() {
    vector<vector<int>> dist = floydWarshall();

    cout << "Матрица кратчайших расстояний:" << endl;
    cout << "  ";
    for (int j = 0; j < V; j++) {
        cout << j << "  ";
    }
    cout << endl;
    cout << "  ";
    for (int j = 0; j < V; j++) {
        cout << "----";
    }
}

```

```

        cout << endl;

        for (int i = 0; i < V; i++) {
            cout << i << " | ";
            for (int j = 0; j < V; j++) {
                if (dist[i][j] == INT_MAX) {
                    cout << "∞ ";
                } else {
                    cout << dist[i][j] << " ";
                }
            }
            cout << endl;
        }
    }
};

```

```

int main() {
    Graph g(4);

    g.addEdge(0, 1, 3);
    g.addEdge(0, 2, 6);
    g.addEdge(1, 2, 2);
    g.addEdge(1, 3, 4);
    g.addEdge(2, 3, 1);
    g.addEdge(3, 2, 1);

    g.printAllPairsShortestPaths();

    return 0;
}

```

Ожидаемый вывод:

Матрица кратчайших расстояний:

```

    0  1  2  3
-----
0 | 0  3  5  7
1 | ∞  0  2  3
2 | ∞  ∞  0  1
3 | ∞  ∞  1  0

```

Пример 3. Восстановление путей (продвинутый уровень)

Задача: Реализовать оба алгоритма с возможностью восстановления кратчайших путей.

```

#include <iostream>
#include <vector>
#include <climits>
#include <algorithm>

using namespace std;

class Graph {
private:
    int V;
    vector<vector<int>>> adj;

public:
    Graph(int vertices) : V(vertices) {
        adj.resize(V, vector<int>(V, 0));
    }

    void addEdge(int u, int v, int weight) {
        adj[u][v] = weight;
        adj[v][u] = weight;
    }

    // Алгоритм Дейкстры с восстановлением пути
    pair<vector<int>, vector<int>> dijkstraWithPath(int start) {
        vector<int> dist(V, INT_MAX);
        vector<int> prev(V, -1);
        vector<bool> visited(V, false);

        dist[start] = 0;

        for (int i = 0; i < V - 1; i++) {
            int u = -1;
            for (int j = 0; j < V; j++) {
                if (!visited[j] && (u == -1 || dist[j] < dist[u])) {
                    u = j;
                }
            }

            if (dist[u] == INT_MAX) break;

            visited[u] = true;

```

```

        for (int v = 0; v < V; v++) {
            if (adj[u][v] != 0 && !visited[v]) {
                if (dist[u] + adj[u][v] < dist[v]) {
                    dist[v] = dist[u] + adj[u][v];
                    prev[v] = u;
                }
            }
        }
    }
}

```

```

return {dist, prev};
}

```

```

vector<int> getPath(int start, int end, const vector<int>& prev) {
    vector<int> path;
    if (prev[end] == -1 && start != end) return path;

    for (int v = end; v != -1; v = prev[v]) {
        path.push_back(v);
    }
    reverse(path.begin(), path.end());
    return path;
}

```

// Алгоритм Флойда-Уоршелла с восстановлением путей

```

pair<vector<vector<int>>, vector<vector<int>>>
floydWarshallWithPath() {
    vector<vector<int>> dist(V, vector<int>(V, INT_MAX));
    vector<vector<int>> next(V, vector<int>(V, -1));

    for (int i = 0; i < V; i++) {
        dist[i][i] = 0;
        for (int j = 0; j < V; j++) {
            if (adj[i][j] != 0) {
                dist[i][j] = adj[i][j];
                next[i][j] = j;
            }
        }
    }

    for (int k = 0; k < V; k++) {

```

```

        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != INT_MAX && dist[k][j] != INT_MAX) {
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                        next[i][j] = next[i][k];
                    }
                }
            }
        }
    }

    return {dist, next};
}

vector<int> getPathFloyd(int u, int v, const vector<vector<int>>& next) {
    if (next[u][v] == -1) return {};

    vector<int> path = {u};
    while (u != v) {
        u = next[u][v];
        path.push_back(u);
    }
    return path;
}

};

int main() {
    Graph g(5);

    g.addEdge(0, 1, 2);
    g.addEdge(0, 2, 5);
    g.addEdge(1, 2, 1);
    g.addEdge(1, 3, 6);
    g.addEdge(2, 3, 2);
    g.addEdge(2, 4, 3);
    g.addEdge(3, 4, 1);

    cout << "=== Алгоритм Дейкстры ===" << endl;
    auto [dist, prev] = g.dijkstraWithPath(0);

    cout << "Расстояния от вершины 0:" << endl;

```

```

for (int i = 0; i < 5; i++) {
    cout << " До " << i << ": " << dist[i] << " (путь: ";
    vector<int> path = g.getPath(0, i, prev);
    for (int v : path) cout << v << " ";
    cout << ")" << endl;
}

cout << "\n=== Алгоритм Флойда-Уоршелла ===" << endl;
auto [distFW, next] = g.floydWarshallWithPath();

cout << "Кратчайшие пути между всеми парами:" << endl;
for (int i = 0; i < 5; i++) {
    for (int j = i + 1; j < 5; j++) {
        cout << " " << i << " → " << j << ": расстояние = " << distFW[i][j];
        vector<int> path = g.getPathFloyd(i, j, next);
        cout << ", путь: ";
        for (int v : path) cout << v << " ";
        cout << endl;
    }
}

return 0;
}

```

Ожидаемый вывод:

=== Алгоритм Дейкстры ===

Расстояния от вершины 0:

До 0: 0 (путь: 0)
 До 1: 2 (путь: 0 1)
 До 2: 3 (путь: 0 1 2)
 До 3: 5 (путь: 0 1 2 3)
 До 4: 6 (путь: 0 1 2 4)

=== Алгоритм Флойда-Уоршелла ===

Кратчайшие пути между всеми парами:

0 → 1: расстояние = 2, путь: 0 1
 0 → 2: расстояние = 3, путь: 0 1 2
 0 → 3: расстояние = 5, путь: 0 1 2 3
 0 → 4: расстояние = 6, путь: 0 1 2 4
 1 → 2: расстояние = 1, путь: 1 2
 1 → 3: расстояние = 3, путь: 1 2 3
 1 → 4: расстояние = 4, путь: 1 2 4

2 → 3: расстояние = 2, путь: 2 3
 2 → 4: расстояние = 3, путь: 2 4
 3 → 4: расстояние = 1, путь: 3 4

Индивидуальные задания

Часть 1. Алгоритм Дейкстры

Каждый вариант содержит задание реализовать алгоритм Дейкстры с указанными особенностями:

№ вар.	Способ реализации	Особенности
1	Без оптимизации ($O(V^2)$)	Неориентированный граф
2	С priority_queue ($O((V+E) \log V)$)	Неориентированный граф
3	Без оптимизации ($O(V^2)$)	Ориентированный граф
4	С priority_queue ($O((V+E) \log V)$)	Ориентированный граф
5	Без оптимизации ($O(V^2)$)	С восстановлением пути
6	С priority_queue ($O((V+E) \log V)$)	С восстановлением пути
7	Без оптимизации ($O(V^2)$)	Разреженный граф ($E \ll V^2$)
8	С priority_queue ($O((V+E) \log V)$)	Плотный граф ($E \approx V^2$)
9	Без оптимизации ($O(V^2)$)	Поиск пути между двумя вершинами
10	С priority_queue ($O((V+E) \log V)$)	Поиск пути между двумя вершинами
11	Без оптимизации ($O(V^2)$)	Нахождение всех кратчайших путей
12	С priority_queue ($O((V+E) \log V)$)	Нахождение всех кратчайших путей

№ вар.	Способ реализации	Особенности
13	Без оптимизации ($O(V^2)$)	Граф с большим количеством вершин ($V=1000$)
14	С priority_queue ($O((V+E) \log V)$)	Граф с большим количеством вершин ($V=1000$)
15	Оба варианта	Сравнение производительности

Часть 2. Алгоритм Флойда-Уоршелла

Каждый вариант содержит задание реализовать алгоритм Флойда-Уоршелла с указанными особенностями:

№ вар.	Особенности	Дополнительное задание
1	Неориентированный граф	Вывод матрицы расстояний
2	Ориентированный граф	Вывод матрицы расстояний
3	Неориентированный граф	Восстановление путей
4	Ориентированный граф	Восстановление путей
5	Неориентированный граф	Обнаружение отрицательных циклов
6	Ориентированный граф	Обнаружение отрицательных циклов
7	Неориентированный граф	Поиск диаметра графа
8	Ориентированный граф	Поиск эксцентриситета вершин
9	Неориентированный граф	Нахождение центра графа
10	Ориентированный граф	Нахождение радиуса графа
11	Неориентированный граф	Сравнение с Дейкстрой ($V=50$)

№ вар.	Особенности	Дополнительное задание
12	Ориентированный граф	Сравнение с Дейкстрой (V=50)
13	Неориентированный граф	Вычисление транзитивного замыкания
14	Ориентированный граф	Вычисление транзитивного замыкания
15	Оба варианта	Сравнение производительности (V=100)

Часть 3. Сравнительный анализ

Задание для всех вариантов:

1. Реализовать граф на 20 вершин со случайными весами рёбер
2. Выполнить алгоритм Дейкстры для случайной стартовой вершины
3. Выполнить алгоритм Флойда-Уоршелла
4. Сравнить результаты (расстояния между одними и теми же парами)
5. Измерить время выполнения для $V = 10, 20, 50, 100$ (для Дейкстры - из одной вершины, для Флойда-Уоршелла - между всеми)
6. Построить график зависимости времени от V

Контрольные вопросы

1. Какие условия должны выполняться для корректной работы алгоритма Дейкстры?
2. Почему алгоритм Дейкстры не работает с отрицательными весами?
3. Какова временная сложность алгоритма Дейкстры с приоритетной очередью?
4. В чём заключается идея алгоритма Флойда-Уоршелла?
5. Какова временная сложность алгоритма Флойда-Уоршелла?
6. Как алгоритм Флойда-Уоршелла обнаруживает отрицательные циклы?
7. В каких случаях предпочтительнее использовать алгоритм Дейкстры?
8. В каких случаях предпочтительнее использовать алгоритм Флойда-Уоршелла?

9. Как восстановить кратчайший путь после выполнения алгоритма Дейкстры?
10. Как восстановить кратчайший путь после выполнения алгоритма Флойда-Уоршелла?
11. Что такое релаксация в контексте алгоритмов поиска кратчайших путей?
12. Как можно оптимизировать алгоритм Дейкстры для разреженных графов?
13. Какую память требует алгоритм Флойда-Уоршелла?
14. При каком количестве вершин алгоритм Флойда-Уоршелла становится неприменимым?
15. Как модифицировать алгоритм Дейкстры для нахождения второго кратчайшего пути?

Требования к отчёту

Титульный лист

Цель работы

Теоретическая часть (алгоритмы Дейкстры и Флойда-Уоршелла)

Постановка задачи (согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода)

Анализ сложности алгоритмов

Сравнительная таблица производительности

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий

Критерии оценивания

Максимальный балл: 100

№	Критерий	Баллы
1	Алгоритм Дейкстры	35
1.1	Реализация алгоритма	15
1.2	Корректность работы	10
1.3	Восстановление пути (по варианту)	10
2	Алгоритм Флойда-Уоршелла	40

№	Критерий	Баллы
2.1	Реализация алгоритма	15
2.2	Корректность работы	10
2.3	Восстановление путей	10
2.4	Обнаружение отрицательных циклов (по варианту)	5
3	Сравнительный анализ	15
3.1	Измерение времени выполнения	5
3.2	Таблица/график сравнения	5
3.3	Выводы о производительности	5
4	Оформление и отчёт	10
4.1	Код отформатирован, есть комментарии	5
4.2	Ссылка на Git-репозиторий	5
Итого		100

Штрафные баллы

Нарушение	Штраф
Неправильная обработка недостижимых вершин	-10
Неверная сложность алгоритма	-10
Отсутствие восстановления путей (по требованию)	-10
Нет сравнительного анализа	-10
Нет ссылки на Git-репозиторий	-10

Нарушение	Штраф
Программа не компилируется	работа не принимается

Итоговая шкала

Оценка	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60